

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING(DATA SCIENCE)
ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Credit Card Fraud Detection Using Anomaly Detection Techniques

SUBMITTED BY:

1. AMAN KUMAR (Roll No: 2400321540028)

Degree: Bachelor of Technology (B.Tech)

CSE-DS

Academic Year: 2026 - 2027

UNDER THE GUIDANCE OF:

Sir Ankit Mishra

Trainer From HCL Team

Status: 3rd Year Project

Date of Submission: 01-10-2026

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	28-Sep-2026	Initial Draft & Scope Definition	Aman Kumar	Ankit Sir
1.0	01-Oct-2026	Final Project Preparation	Aman Kumar	HCL Committee

Guidance Note: This Software Requirements Specification document conforms to the IEEE 830-1998 standard recommended practice.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance targets, and constraints of the proposed system: **'Credit Card Fraud Detection Using Anomaly Detection Techniques'**. This specification serves as a binding reference agreement between student developers, project evaluators, academic supervisors, and prospective end users during the 3rd-year engineering curriculum.

1.2 Scope of the System

The proposed application is designed to solve *the problem of identifying fraudulent credit card transactions in a highly imbalanced dataset (fewer than 0.2% fraud)* by delivering an end-to-end machine learning platform. It trains and compares four models (Logistic Regression, Random Forest, XGBoost, and Isolation Forest), produces a fraud probability and risk level for every transaction, stores results in a SQL database, and exposes them through a REST API and an interactive Transaction Risk Dashboard. Key boundary guidelines include:

- **In-Scope Capabilities:** Dataset ingestion and preprocessing (cleaning, scaling, stratified splitting); class-imbalance handling (class weights, SMOTE, undersampling, threshold tuning); model training, tuning and comparison; evaluation using Precision, Recall, F1-score, ROC-AUC, PR-AUC and confusion matrices; single and batch fraud prediction with risk scoring; SQL storage of transactions, predictions and model metrics; and a web dashboard with charts, filters and a live prediction form.
- **Out-of-Scope Boundaries:** Integration with live banking or payment-processor systems, real-time streaming (e.g., Kafka), blocking or reversing real transactions, user authentication and role management for the dashboard, and cloud/CI-CD deployment. These are listed as future improvements.
- **Expected Benefits:** An objective, metric-driven comparison of supervised and unsupervised fraud detectors; a ranked, explainable risk score that helps analysts prioritise suspicious transactions; a reproducible pipeline (fixed seeds, saved models, requirements file); and immediate browser-based access to results.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Standard Context
Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Architecture	API	Application Programming Interface for data interchange via JSON over HTTP(S).
Network	REST	Representational State Transfer, a stateless architectural style for web services.
Database	SQL / ORM	Structured Query Language; Object-Relational Mapping (SQLAlchemy) used to access the database from Python.
Database	CRUD	Create, Read, Update, Delete basic data manipulation operations.
Project Specific	Anomaly Detection	Identifying rare observations that differ significantly from the majority of the data.
Project Specific	SMOTE	Synthetic Minority Over-sampling Technique; creates synthetic minority-class samples to balance training data.
Project Specific	Isolation Forest	Unsupervised anomaly detector that isolates outliers using random tree partitioning; outputs an anomaly score rather than a probability.
Project Specific	XGBoost	Gradient-boosted decision tree library, typically strong on tabular data.
Project Specific	PCA	Principal Component Analysis; features V1-V28 in the dataset are anonymised PCA components.
Metrics	ROC-AUC / PR-AUC	Area under the Receiver Operating Characteristic curve / Precision-Recall curve. PR-AUC is more informative on highly imbalanced data.
Metrics	TP / FP / TN / FN	True/False Positives and Negatives from the confusion matrix.

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.
- **Dataset:** Credit Card Fraud Detection, ULB Machine Learning Group, Kaggle (<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>).
- **SMOTE:** N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique", Journal of Artificial Intelligence Research, 2002.
- **Isolation Forest:** F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation Forest", IEEE International Conference on Data Mining (ICDM), 2008.
- **XGBoost:** T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System", ACM KDD, 2016.
- **Official documentation:** scikit-learn, imbalanced-learn, Flask / FastAPI, SQLAlchemy, Chart.js and Plotly.js.

1.5 Document Overview

The remainder of this SRS is partitioned into structural sections: Section 2 outlines the product perspective, target user personas, operational context, and design constraints. Section 3 itemizes detailed functional requirements organized into logical software modules. Section 4 specifies external interfaces (UI, Hardware, Software, Communication) including the REST API. Section 5 defines non-functional attributes (Performance, Reliability, Security and others). Section 6 provides standard academic modeling diagrams (use case, DFD, ERD and sequence). Section 7 is the academic sign-off.

2. Overall Description

2.1 Product Perspective

The project is structured as a self-contained multi-tier system comprising (i) an offline machine learning pipeline that preprocesses data, trains and evaluates models and saves them as .pkl files; (ii) an application/API tier (Flask or FastAPI) that loads the saved models and serves predictions; (iii) a SQL database (SQLite for development, PostgreSQL/MySQL for production) that persists transactions, predictions and model metrics; and (iv) a presentation tier, an HTML/CSS/JavaScript dashboard using Chart.js/Plotly. Communication between tiers uses RESTful JSON endpoints.

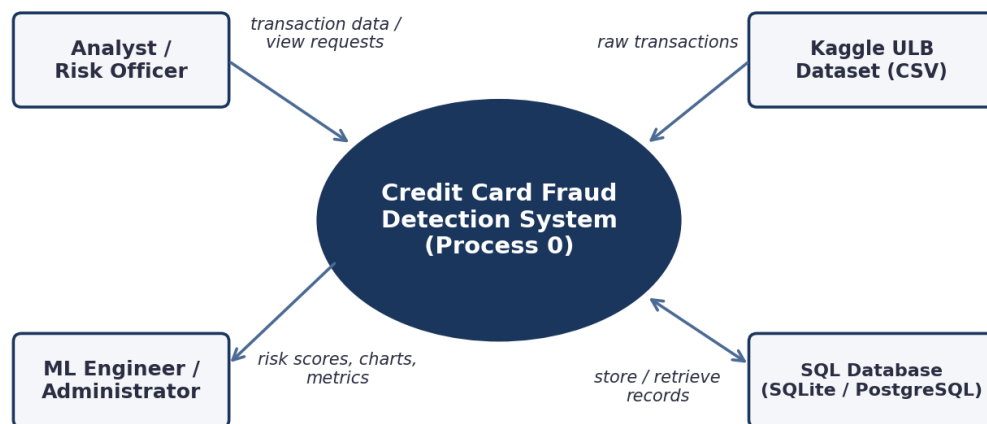


Figure 2.1 Context Diagram (DFD Level 0)

2.2 User Classes and Characteristics

The system accommodates distinct user personas. As user authentication is outside the current scope, access is not role-restricted in the first release; the roles below describe intended usage.

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
ML Engineer / Administrator	High (Python, ML, SQL)	Prepares the dataset, runs preprocessing, training and evaluation scripts, tunes thresholds, initialises the database, and deploys the application.

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
Fraud Analyst / Risk Officer	Intermediate (general web user)	Uses the dashboard to review KPIs, risk distribution and flagged transactions; submits single or batch transactions for scoring; compares model metrics.
External API Client	High (developer / system)	Calls the REST endpoints programmatically to obtain fraud probability and risk level for transactions.

2.3 Operating Environment

- **Client Requirements:** Modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari); minimum display resolution 1366x768 for desktop, with a responsive layout for smaller screens.
- **Server Runtime Environment:** Windows, Linux (e.g., Ubuntu 22.04 LTS) or macOS; Python 3.9 or later; Flask or FastAPI (one to be chosen) with Uvicorn if FastAPI is used.
- **ML Libraries:** pandas, NumPy, scikit-learn, XGBoost, imbalanced-learn, joblib, matplotlib, seaborn, Plotly and (optionally) SHAP.
- **Database Storage Tier:** SQLite for development; PostgreSQL or MySQL for production, accessed through SQLAlchemy.
- **Recommended Hardware:** Multi-core CPU and at least 8 GB RAM for training on the 284,807-row dataset with SMOTE and cross-validation; no GPU required.

2.4 Design and Implementation Constraints

- **Academic Budget Constraint:** Only free and open-source tools are used; development runs locally with SQLite, with free-tier hosting as an optional extension.
- **Data Constraints:** The dataset has only about 0.172% fraudulent records (492 of 284,807). Features V1-V28 are anonymised PCA components, so original business attributes are unavailable; only Time and Amount are in their original form.
- **Methodological Constraints:** Accuracy shall not be the main evaluation metric. SMOTE shall be applied to the training set only, after the train/test split, to avoid data leakage. Splits shall be stratified.
- **Modularity Standards:** Code is organised into data_preprocessing, train, evaluate and predict modules (src/), and an application package (app/) following a modular REST service pattern with clear separation of concerns.
- **Reproducibility:** Fixed random seeds, persisted models and a requirements.txt file are mandatory.

2.5 Assumptions and Dependencies

- **Assumptions:** The ULB creditcard.csv dataset is available and placed in data/raw/. Incoming transactions supplied to the API contain the same 30 features (Time, V1-V28, Amount) as the training data. Users have a modern browser and local network access to the server.
- **Dependencies:** Availability of the Kaggle dataset; third-party Python libraries listed in requirements.txt; the Chart.js / Plotly.js libraries for dashboard visualisation; and a supported SQL database engine.

- **Limitation:** The data covers two days of European card transactions from September 2013; model performance on other populations or later periods may differ (concept drift).

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x):

3.1 Module 1: Data Ingestion and Preprocessing

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	Dataset Loading	Read creditcard.csv from data/raw/; verify the presence of Time, V1-V28, Amount and Class columns.	High
FR-1.2	Data Cleaning	Detect missing values; remove duplicate rows; report class distribution and imbalance ratio.	High
FR-1.3	Feature Scaling	Scale Amount and Time using StandardScaler or RobustScaler; persist the fitted scaler (scaler.pkl) for inference.	High
FR-1.4	Stratified Train/Test Split	Split data (e.g., 80/20) so that fraud cases appear in both sets; fixed random seed.	High
FR-1.5	Exploratory Data Analysis	Generate class distribution, Amount/Time distributions for fraud vs. legitimate, correlation heatmap and outlier analysis.	Medium

3.2 Module 2: Model Training and Evaluation

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Multi-Model Training	Train Logistic Regression, Random Forest, XGBoost (supervised) and Isolation Forest (unsupervised).	High
FR-2.2	Class Imbalance Handling	Apply class_weight='balanced' (LR, RF), scale_pos_weight (XGBoost), optional SMOTE (training set only) and random undersampling baseline.	High
FR-2.3	Hyperparameter Tuning	Use GridSearchCV / RandomizedSearchCV with Stratified K-Fold cross-validation, scored on PR-AUC or F1 (not accuracy).	Medium
FR-2.4	Isolation Forest Score Normalisation	Min-max normalise anomaly scores to 0-1; set contamination close to the real fraud rate (about 0.0017).	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.5	Metric Computation	Compute Precision, Recall, F1-score, ROC-AUC, PR-AUC and confusion matrix at a configurable decision threshold; support threshold tuning.	High
FR-2.6	Visual Report Generation	Save ROC curves, Precision-Recall curves, confusion-matrix heatmaps, feature-importance charts and a metrics comparison bar chart to reports/figures/.	Medium
FR-2.7	Model Persistence	Save trained models and scaler as .pkl files with joblib in models/; record metrics in the model_metrics table.	High
FR-2.8	Explainability	Report feature importance per model; optional SHAP values for individual predictions.	Low

3.3 Module 3: Prediction and Risk Scoring

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Single Transaction Prediction	POST /api/predict with model name and 30 features (Time, V1-V28, Amount); returns probability, prediction and risk level.	High
FR-3.2	Batch Prediction	POST /api/predict/batch accepting a JSON array or CSV file of transactions; returns a score for each row.	High
FR-3.3	Risk Level Classification	Map fraud probability to Low / Medium / High / Critical bands as per the table below.	High
FR-3.4	Configurable Thresholds	Risk band boundaries and the decision threshold are configurable in src/predict.py.	Medium
FR-3.5	Input Validation	Reject requests with missing/non-numeric features or unknown model names with a clear error message and HTTP 4xx status.	High

Fraud Probability	Risk Level	Colour Indicator
0.00 - 0.30	Low	Green
0.30 - 0.60	Medium	Yellow
0.60 - 0.85	High	Orange
0.85 - 1.00	Critical	Red

3.4 Module 4: Data Storage

Req ID	Feature Description	Input / Validation Rule	Priority
FR-4.1	Transaction Storage	Persist submitted/loaded transactions (amount, time, V1-V28, optional actual_class) in the transactions table.	High
FR-4.2	Prediction Storage	Persist model name, fraud probability, predicted class, risk level and timestamp in predictions with a foreign key to transactions.	High
FR-4.3	Model Run Metrics Storage	Persist Precision, Recall, F1, ROC-AUC and PR-AUC per training run in model_metrics.	Medium
FR-4.4	Analytical Queries	Provide SQL queries for top-risk transactions, count per risk level, average amount by class and latest metrics per model.	Medium

3.5 Module 5: Dashboard, Reporting and Monitoring

Req ID	Feature Description	Input / Validation Rule	Priority
FR-5.1	KPI Cards	Show total transactions, flagged frauds, fraud rate and total amount at risk (via GET /api/stats).	High
FR-5.2	Model Comparison Chart	Grouped bar chart of Precision, Recall, F1 and ROC-AUC for all models (via GET /api/metrics).	High
FR-5.3	ROC and PR Curves	Interactive curves with all models overlaid.	Medium
FR-5.4	Confusion Matrix View	Heatmap with a model selector dropdown.	Medium
FR-5.5	Risk Distribution Chart	Donut/pie chart of Low / Medium / High / Critical counts.	Medium
FR-5.6	Transaction Risk Table	Sortable and filterable table with fraud probability and colour-coded risk badges (via GET /api/transactions).	High
FR-5.7	Live Prediction Form	Enter or upload a transaction and display an instant risk score.	High
FR-5.8	Time Series and Feature Importance	Fraud activity over time; top contributing features per model.	Low
FR-5.9	Health Check	GET /api/health returns service status for monitoring.	Low

4. External Interface Requirements

4.1 User Interfaces (UI)

The interface follows standard UX usability best practices (clean navigation, consistent layout, colour-coded risk badges). All forms shall provide immediate visual feedback (success banners or inline error alerts under erroneous fields). The dashboard consists of the following screens/panels:

Screen / Panel	Main Components	Data Source
Overview	KPI cards, risk distribution donut chart, fraud-over-time chart	GET /api/stats
Model Comparison	Grouped metric bars, ROC / PR curves, confusion matrix with model selector, feature importance	GET /api/metrics
Transaction Risk Table	Sortable, filterable table with probability and colour-coded risk badge	GET /api/transactions
Live Prediction	Input form / file upload, model selector, result card with probability and risk level	POST /api/predict, /api/predict/batch

Wireframes: Insert Figma wireframes or screenshots of the dashboard here after implementation: (1) Overview, (2) Model Comparison, (3) Transaction Risk Table / Live Prediction.

4.2 Hardware Interfaces

Standard Deployment: Standard input/output peripherals (keyboard, mouse, display of at least 720p). The system has no dedicated hardware interface and does not require a GPU or embedded devices.

4.3 Software Interfaces

- **ML Libraries:** scikit-learn, XGBoost and imbalanced-learn for modelling; joblib for loading and saving .pkl model files.
- **Database Connector:** SQLAlchemy ORM interfacing with SQLite (development) or PostgreSQL/MySQL (production). Schema is defined in sql/schema.sql.
- **Front-end Libraries:** Chart.js and/or Plotly.js for charts, consuming the JSON endpoints below.

REST API Summary

Method	Endpoint	Description
GET	/	Serves the dashboard
POST	/api/predict	Predict fraud for a single transaction
POST	/api/predict/batch	Predict fraud for multiple transactions (JSON/CSV)

Method	Endpoint	Description
GET	/api/metrics	Return evaluation metrics for all models
GET	/api/transactions	List stored transactions with risk scores
GET	/api/stats	Summary statistics (totals, fraud rate, risk distribution)
GET	/api/health	Health check

Example: Single Prediction

```
POST /api/predict
{ "model": "xgboost",
  "features": { "Time": 406, "Amount": 149.62,
               "V1": -1.36, "V2": -0.07, ..., "V28": -0.02 } }

Response:
{ "model": "xgboost", "fraud_probability": 0.9231,
  "prediction": "Fraud", "risk_level": "Critical" }
```

4.4 Communication Interfaces

- **Web Traffic Protocol:** HTTP on localhost during development (port 5000 for Flask, 8000 for FastAPI); HTTPS with TLS shall be used for any deployed instance.
- **Data Serialization:** Payloads are exchanged as UTF-8 encoded application/json; batch prediction additionally accepts CSV file upload (multipart/form-data).

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification & Testing Method
5.1 Performance	Single prediction API response under 500 ms; dashboard initial load under 3 s on a standard laptop; batch scoring of 10,000 rows within 30 s.	Timing tests with pytest and Apache Benchmark / JMeter.
5.2 Model Quality	Evaluate on a stratified hold-out set using Precision, Recall, F1, ROC-AUC and PR-AUC; accuracy is not the primary metric; no data leakage (SMOTE only on training data).	Review of src/evaluate.py outputs; cross-validated PR-AUC; confusion matrices.
5.3 Security	Parameterised queries through the ORM to prevent SQL injection; strict input validation and file-type checks on uploads; secrets and DB URLs kept in environment variables (.env); HTTPS for deployed instances.	Code review; invalid-input tests; OWASP ZAP baseline scan.
5.4 Reliability	Graceful error responses (HTTP 4xx/5xx with messages) instead of crashes; /api/health endpoint; database writes fail safely without corrupting existing records.	Negative test cases in tests/test_api.py; manual fault injection.

Attribute	Target Metric / Standard	Verification & Testing Method
5.5 Reproducibility	Fixed random seeds, saved models and scaler, pinned requirements.txt so that results can be regenerated.	Re-running train.py and evaluate.py yields the same metrics.
5.6 Maintainability	Modular code (src/ and app/), documented functions, SQL schema and queries kept in sql/, unit and API tests with pytest.	Code review; pytest test suite.
5.7 Usability	Clear colour-coded risk indicators, sortable/filterable tables, inline form validation messages.	User walkthrough with supervisor/peers.
5.8 Portability	Runs on Windows, Linux and macOS with Python 3.9+; dashboard works on Chrome, Edge, Firefox and Safari with a responsive layout.	Cross-browser check; run setup steps on two operating systems.

6. System Design and Analysis Models

University evaluation rubrics require standard graphical analysis models illustrating the dynamic and static behavior of the application. The Level 0 context diagram is given in Figure 2.1; the remaining models follow.

6.1 Use Case Diagram

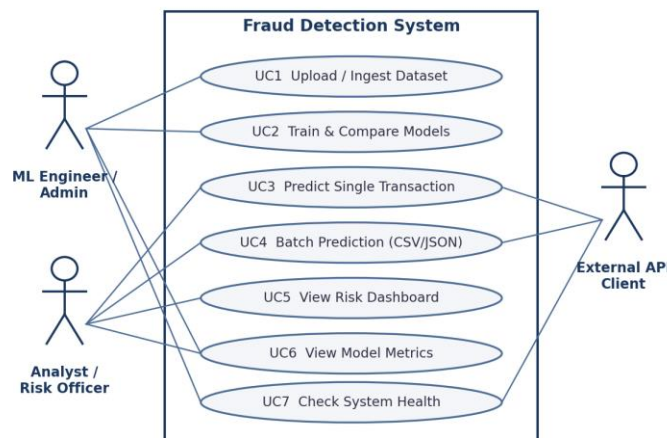


Figure 6.1 Use Case Diagram

ID	Use Case	Primary Actor	Brief Description
UC1	Upload / Ingest Dataset	ML Engineer	Loads creditcard.csv, validates columns, cleans and stores data.
UC2	Train & Compare Models	ML Engineer	Trains four models, computes metrics, saves models and metrics.

ID	Use Case	Primary Actor	Brief Description
UC3	Predict Single Transaction	Analyst / API Client	Returns fraud probability and risk level for one transaction.
UC4	Batch Prediction	Analyst / API Client	Scores a CSV/JSON set of transactions and stores the results.
UC5	View Risk Dashboard	Analyst	Views KPIs, charts and the filterable transaction risk table.
UC6	View Model Metrics	Analyst / ML Engineer	Compares models via metric charts, curves and confusion matrices.
UC7	Check System Health	ML Engineer / API Client	Calls /api/health to confirm the service is running.

6.2 Data Flow Diagram (Level 1)

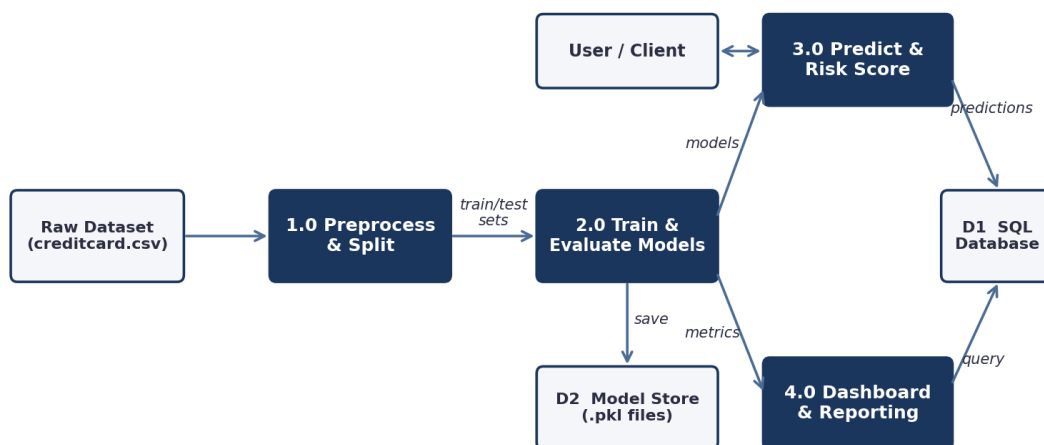


Figure 6.2 Data Flow Diagram (Level 1)

6.3 Entity-Relationship Diagram and Data Dictionary

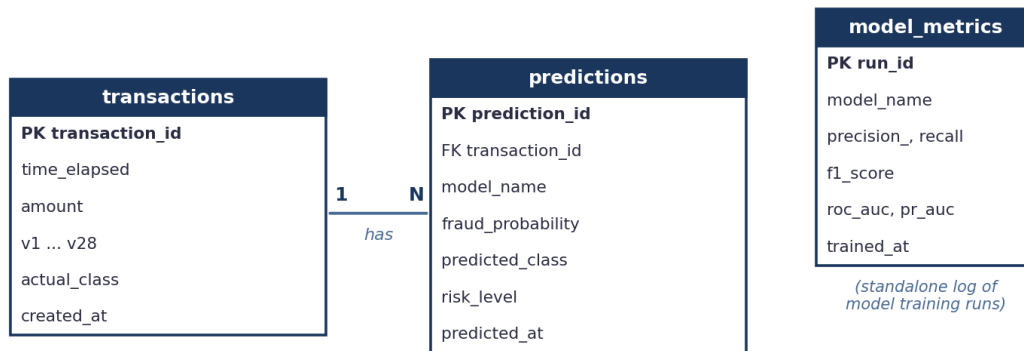


Figure 6.3 Entity-Relationship Diagram

Table	Key Columns	Notes
transactions	transaction_id (PK), time_elapsed, amount, v1..v28, actual_class, created_at	Stores each transaction; actual_class holds ground truth when known.
predictions	prediction_id (PK), transaction_id (FK), model_name, fraud_probability, predicted_class, risk_level, predicted_at	One row per transaction per model; risk_level restricted to Low / Medium / High / Critical.
model_metrics	run_id (PK), model_name, precision_, recall, f1_score, roc_auc, pr_auc, trained_at	One row per model training run.

The schema is in 3NF: each non-key attribute depends only on its table's primary key, and one transaction may have many predictions (1:N), one for each model that scored it.

6.4 Sequence of Messages: Single Prediction

Step	From	To	Message / Action
1	Analyst	Dashboard (UI)	Enters transaction features, selects model and clicks Predict.
2	Dashboard (UI)	API (/api/predict)	Sends JSON request with model name and features.
3	API	Validator	Checks that all 30 features are numeric and the model name is valid; on failure returns an error (alternate flow).
4	API	Model Store	Loads scaler.pkl and the selected model .pkl; scales Time and Amount.

Step	From	To	Message / Action
5	API	Model	Computes fraud probability; maps to prediction and risk level.
6	API	SQL Database	Inserts transaction and prediction records.
7	API	Dashboard (UI)	Returns JSON with fraud_probability, prediction and risk_level.
8	Dashboard (UI)	Analyst	Displays result card with colour-coded risk badge and refreshes charts.

7. Academic Review & Sign-Off

This Software Requirements Specification document has been submitted and reviewed by the academic supervisory panel:

Project Coordinator / Guide

Signature: _____

Date: _____

Internal Examiner

Signature: _____

Date: _____

Head of Department (HOD)

Signature: _____

Date: _____